

Tutorial DSDV Lab

This lab explores Destination-Sequenced Distance-Vector routing while one relay moves out of radio range and back. You will build the supplied INET model, observe proactive table updates and reconvergence in Qtenv, and analyze delivery, delay, hop count, and interruption time. The same ten-node geometry supports comparison with AODV.

1 Prepare the workspace

Use an OMNeT++ installation with a compatible INET Framework installation. INET must be visible as an open project in the same IDE workspace. This lab uses INET MANET routers, DSDV, wireless radios, PingApp, and canvas visualizers. Keep the supplied AodvMobilityLab-main files together because the DSDV network extends the shared topology.

1. Start the OMNeT++ IDE and choose a workspace. Confirm that the inet project is open and builds successfully.
2. Create an OMNeT++ project named DsdvMobilityLab or use the existing AodvMobilityLab project. Copy the supplied NED, C++, and ini files into the project root.
3. Confirm that AodvMobilityNetwork.ned defines both AodvProtocolNetwork and DsdvProtocolNetwork. Keep the shared ManetMetrics and visualizer files on the NED and C++ paths.

If you use a subfolder, set it as both the NED source folder and C++ source location. Point the run configuration working directory and ini file to that folder.
4. Add the inet project reference and refresh the generated Makefile. The DSDV configuration uses INET's existing Dsdv routing module.
5. Build the project and resolve compile or NED errors before running DSDV.

2 Import an existing OMNeT++ project

The supplied AodvMobilityLab-main folder is a source folder without a .project file. Create a project, copy the files, reference inet, and verify that the NED source folder includes AodvMobilityNetwork.ned and ManetMetrics.ned. DsdvProtocolNetwork is selected from the shared omnetpp.ini.

3 Understand what the files build

File or group	Role in the simulation
AodvMobilityNetwork.ned	Defines the shared ten-node topology plus AODV and DSDV network variants.
omnetpp.ini	Sets protocol parameters, radio range, traffic timing, mobility, visualization, and result recording.
ManetMetrics.cc/.h/.ned	Computes destination delivery, one-way delay, hop count, PDR, and reception gap.
AnimationController and visualizer files	Control movement trails, accepted-hop arrows, labels, route overlays, and selected drops.
Makefile and project files	Build the C++ modules and connect the project to INET.

The topology is deliberately sparse. With a 250 m communication range, the initial central path can use host[0] -> host[1] -> host[2] -> host[3] -> host[9] (four links). An upper route can use host[0] -> host[4] -> host[5] -> host[6] -> host[3] -> host[9] (five links). DSDV maintains routes before data is sent; it does not use AODV RREQ/RREP/RERR discovery.

4 Run DSDV

1. Open omnetpp.ini and select the DsdvMobilityDemo configuration. Run with Qtenv so orange destination updates, black ping routes, drops, and the gold relay trail are visible.

2. Set the working directory to the folder containing omnetpp.ini. Run to the 60 s limit and pause around the observation times.

3. Watch host[0], host[9], and host[2]. DSDV sends periodic advertisements from every node; the orange filter highlights destination-origin information so one update propagation can be followed without covering the canvas.

Parameter guide. sim-time-limit = 60 s and seed-set = 41 control observation length and repeatability. communicationRange = 250 m controls wireless reachability. DSDV helloInterval = 2 s controls advertisement frequency, routeLifetime = 5 s controls how long a route remains usable without fresh information, maxVariance = 0.15 limits accepted metric variation, and broadcastDelay = uniform(0 s, 0.02 s) spreads rebroadcasts. PingApp starts at 10 s to allow tables to converge, sends 256 B packets every 0.6 s, and uses hopLimit = 32. Changing update frequency trades fresher routes for more control traffic.

What the custom code measures

ManetMetrics subscribes to PingApp signals and destination-side packet activity. It counts delivered requests, computes one-way delay from packet creation, estimates hop count from the HopLimitInd tag, and records delivery ratio and the longest reception gap. The visualizer highlights selected DSDV advertisements and the current ping path; an orange arrow is one accepted hop of an update, not the complete route. Summary scalars are written when finish() runs.

Recording guide. scalar-recording is enabled for summary values. vector-recording is disabled globally, then enabled for PingApp RTT, metrics.destinationDelay, and metrics.destinationHopCount. Enable additional routing-table or packet vectors only for a focused experiment; proactive protocols can produce large traces because every node advertises periodically.

5 Observe the route change

Simulation time	What to inspect
0-2 s AODV / 0-10 s DSDV	AODV discovers on demand; DSDV uses this quiet period for proactive table convergence.
18-24 s	The relay moves from (520,325) to (520,500). Watch the central route break and protocol-specific recovery.
24-36 s	The relay remains away. Check whether the upper bypass carries data and how hop count changes.
36-42 s	The relay returns. Observe AODV rediscovery or DSDV reconvergence.
After recovery	Finish the run and compare scalars, selected vectors, and event-log messages.

DSDV has no RREQ, RREP, or RERR exchange. When the relay moves, nodes continue advertising sequence-numbered routes; stale entries expire and newer information reconverges through the bypass. Black depicts ping traffic, orange shows selected destination-origin updates, red DROP marks selected failures, and gold traces relay movement. Animation speed changes viewing time only.

6 Find and analyze results

1. After the run finishes, refresh Project Explorer and open the results folder. OMNeT++ writes .sca scalars and selected .vec vectors; an .elog event log is also recorded.
2. Filter on metrics and host[0].app[0]. PingApp. Compare sent, received, lost, delivery ratio, destination delay, destination hops, RTT, and longest reception gap.
3. Plot metrics.destinationHopCount and metrics.destinationDelay against simulation time. Plot RTT beside them. Use the event log to align drops with route expiry and advertisement propagation.
4. Repeat with a different helloInterval, routeLifetime, maxVariance, or communicationRange. Record all parameter values. For comparison with AODV, align traffic start, run length, range, packet size, recording settings, and seeds.

Recorded result	How to interpret it
PingApp sent/received/lost	Offered requests and end-to-end reply outcomes; replies can fail independently of requests.
destination PDR	Destination deliveries divided by request attempts. Record the traffic window beside it.
destination delay and hops	One-way delay and estimated links for delivered requests only.
RTT vector	Round-trip time measured by PingApp; compare spikes with route changes.
longest reception gap	Largest interval without a destination request arrival during the run.
protocol parameters	Range, timers, traffic settings, run length, seed, and recording flags define the experiment.
AODV messages	RREQ, RREP, and RERR arrows show on-demand discovery and repair.
DSDV updates	Orange selected advertisements show proactive information propagation; every node still updates periodically.

Use destination delivery and PingApp replies together. A request can reach host[9] while its reply fails on the return path, so the two counts can differ. A longer reception gap may reflect route expiry plus advertisement convergence; use update events and vector sample times to explain it.

7 Student analysis questions

- Identify the first successful DSDV path and record its hop count. How long did the proactive convergence period last before the first delivery?
- Around relay departure, record the last pre-break arrival and first later arrival. Compare the difference with the longest reception gap.
- Change helloInterval and routeLifetime separately. Predict the effects on stale-route duration, recovery time, and control traffic.
- Repeat with a smaller communicationRange or larger broadcastDelay. Explain the expected effect on reconvergence, drops, delay, and vector size.
- Compare DSDV's periodic destination updates with AODV's RREQ/RREP/RERR exchange using matched traffic windows. Report sent, received, PDR, delay, RTT, longest gap, and all changed parameters.

8 Troubleshooting and limits

- Unknown NED type or missing inet header: confirm that inet is open, built, referenced, and visible in the NED and C++ build paths.
- DSDV does not run: select DsdvMobilityDemo and confirm that DsdvProtocolNetwork is present in AodvMobilityNetwork.ned.
- Only a generic network appears: select DsdvMobilityDemo. The base network does not show the DSDV-specific legend.
- No metrics scalars: let the 60 s simulation finish normally, then refresh the results directory.
- No orange update arrows: use Qtenv, slow the animation, and verify that the DsdvHello visualizer filter is enabled.
- This is an instructional proactive-routing experiment. Quantitative comparisons require aligned INET versions, traffic windows, radio parameters, and multiple seeds.

Reference documentation

OMNeT++ User Guide: <https://doc.omnetpp.org/omnetpp/UserGuide.pdf>

INET installation: <https://inet.omnetpp.org/Installation.html>

INET usage and project references: <https://inet.omnetpp.org/docs/users-guide/ch-usage.html>